A Project Report on

Software as a Service (SaaS) using HDFS

Submitted By

Name: Rohit Sharma

Roll No: 7347

Submitted as a partial fulfillment towards **310254: Cloud Computing**

Of term II of

T.E. (Computer Engineering)

ARMY INSTITUTE OF TECHNOLOGY

Department of Computer Engineering

Savitribai Phule Pune University

2025-26

Onward to Glory

C E R T I F I C A T E

This is to certify that **Rohit Sharma** of Army Institute of Technology, Dighi, Pune has submitted project report on as a partial fulfillment **Cloud Computing** of **Term –II** Third Year of Bachelor of Computer Engineering, from Savitribai Phule Pune University, Pune, during the Academic Year 2025-26.

Ms. Pradnya Tapkir	Dr. S. R. Dhore
Subject Teacher	**H.O.D.**

ACKNOWLEDGMENT

I would like to express my sincere gratitude to Ms. Pradnya Tapkir for providing me with the opportunity to work on the project titled *"Implementation of a Private SaaS Cloud over LAN using HDFS with Encryption."* Their guidance, encouragement, and valuable feedback throughout the development of this project were instrumental in shaping both the technical direction and successful completion of the work.

I am also thankful to the department and institution for providing the necessary infrastructure and laboratory facilities required to implement and test the system in a controlled environment.

I would like to extend my appreciation to my peers and colleagues who offered support, shared insights, and assisted in troubleshooting during various stages of the project.

Finally, I acknowledge the open-source community, particularly the contributors of Apache Hadoop and Hadoop Distributed File System (HDFS), whose tools and documentation played a significant role in the successful realization of this project.

ABSTRACT

Cloud computing has transformed the way data and applications are managed by providing scalable, flexible, and cost-efficient solutions. Among the various service models, Software as a Service (SaaS) enables users to access software applications over a network without requiring local installation. This project focuses on designing and implementing a private SaaS cloud within a Local Area Network (LAN) environment using the Hadoop Distributed File System (HDFS). The system includes a custom-built cloud controller developed using open-source technologies to manage file storage, segmentation, encryption, and retrieval.

The primary functionality of the system involves dividing files into smaller blocks, encrypting each block, and distributing them across multiple nodes in the network. The use of encryption ensures data confidentiality, while HDFS provides fault tolerance and scalability. The system also supports secure file retrieval by reassembling and decrypting the stored blocks.

This project demonstrates the practical implementation of distributed storage and cloud concepts in a controlled lab environment. It provides insights into file distribution strategies, encryption mechanisms, and the working principles of cloud infrastructure, making it a valuable educational tool for understanding real-world cloud systems.

1. INTRODUCTION

Cloud computing is a paradigm that enables on-demand access to computing resources such as storage, processing power, and applications over the internet. Among the different service models, SaaS is widely used due to its ease of deployment and accessibility. Instead of installing software locally, users can access applications through a network interface.

In this project, a private cloud is implemented within a LAN environment to simulate real-world cloud services. Unlike public cloud platforms, which require internet connectivity and significant infrastructure, a LAN-based cloud provides a cost-effective and controlled environment for experimentation and learning.

The system leverages Apache Hadoop, an open-source framework designed for distributed storage and processing of large datasets. The storage component, HDFS, divides files into blocks and distributes them across multiple machines (nodes), ensuring redundancy and reliability.

A custom cloud controller is developed to handle:

- File segmentation
- Encryption and decryption
- Communication with HDFS
- Upload and download operations

This project bridges theoretical cloud computing concepts with practical implementation, enabling a deeper understanding of distributed systems.

2. OBJECTIVES

The main objectives of this project are:

- To design and implement a private SaaS cloud within a LAN
- To develop a custom cloud controller using open-source technologies
- To utilize HDFS for distributed storage
- To implement file segmentation into blocks
- To ensure secure storage using encryption techniques
- To enable reliable upload and download operations
- To simulate real-world cloud infrastructure in a lab environment

3.LITERATURE REVIEW

Cloud computing has evolved significantly over the past decade, with frameworks like Apache Hadoop playing a crucial role in handling large-scale data. HDFS is designed to store massive datasets across distributed nodes, providing high throughput and fault tolerance.

Traditional storage systems rely on centralized architectures, which are prone to failure and scalability limitations. Distributed file systems like HDFS overcome these issues by replicating data across multiple nodes.

Encryption is another critical aspect of cloud systems. Techniques such as Advanced Encryption Standard (AES) are widely used to secure data. By encrypting data before storage, the system ensures confidentiality even if unauthorized access occurs.

Previous research highlights the importance of combining distributed storage with encryption to build secure and efficient cloud systems. This project adopts these principles and implements them in a simplified LAN-based environment.

4.SYSTEM ARCHITECTURE

The proposed system is designed as a private cloud environment within a Local Area Network (LAN), integrating distributed storage and secure data handling. The architecture is built around the Hadoop Distributed File System (HDFS) and a custom-developed cloud controller.

4.1 Components of the System

- **ClientNode:**
  The client node provides an interface through which users can upload and download files. It sends requests to the cloud controller and receives processed data.
- **Controller:**
  This is the core component of the system. It manages file operations such as segmentation, encryption, and communication with the storage layer. It also maintains metadata required for reconstructing files.
- **NameNode (HDFS):**
  In HDFS, the NameNode is responsible for managing metadata, including file structure, block locations, and access details.
- **DataNodes (HDFS):**
  These nodes store the actual file blocks. Each block is distributed across multiple DataNodes to ensure redundancy and fault tolerance.

4.2 System Workflow

File Upload Process:

1. The user selects a file from the client system.
2. The file is sent to the cloud controller.
3. The controller divides the file into smaller blocks.
4. Each block is encrypted for security.
5. Encrypted blocks are stored in HDFS across multiple DataNodes.

6. Metadata such as block order and storage location is recorded.

File Download Process:

1. The user requests a file from the system.
2. The cloud controller retrieves metadata from the NameNode.
3. Encrypted blocks are fetched from DataNodes.
4. Each block is decrypted.
5. Blocks are merged to reconstruct the original file.
6. The file is delivered to the client.

4.3 Key Features of the Architecture

- **Distributed Storage:**
  Data is stored across multiple nodes using HDFS, ensuring scalability.
- **Fault Tolerance:**
  Replication of blocks ensures that data remains available even if some nodes fail.
- **Security:**
  Encryption of file blocks protects data from unauthorized access.
- **Modularity:**
  The separation of client, controller, and storage components allows easy maintenance and upgrades.

5.TECHNOLOGIES USED

The implementation of the system relies on several open-source technologies:

- **Framework:**
  Apache Hadoop is used for managing distributed storage and processing.
- **Storage System:**
  Hadoop Distributed File System (HDFS) is used to store file blocks across nodes.
- **Programming Language:**
  Python / Java (depending on implementation)
- **Encryption Technique:**
  Advanced Encryption Standard (AES) is used to secure file blocks.
- **Networking:**
  Socket programming or REST APIs are used for communication between client and controller.

6.METHODOLOGY

The methodology describes how the system processes and manages files in a secure distributed environment.

6.1 File Segmentation

Files are divided into smaller blocks before storage. This improves:

- Parallel processing
- Efficient storage
- Fault tolerance

6.2 Encryption Process

Each block is encrypted before being stored in HDFS. This ensures that even if data is accessed from storage nodes, it remains unreadable without the decryption key.

6.3 Upload Mechanism

- Input file is received
- File is split into blocks
- Each block is encrypted
- Encrypted blocks are stored in HDFS
- Metadata is recorded

6.4 Download Mechanism

- Metadata is retrieved
- Encrypted blocks are fetched
- Blocks are decrypted
- Original file is reconstructed

7.CODE

7.1 Controller.py

```
from flask import Flask, request, send_file, render_template
import os
import subprocess
import re
from cryptography.fernet import Fernet
app = Flask(__name__)
HDFS_PATH = "/user/hadoop/"
subprocess.run(f'hdfs dfs -mkdir -p "{HDFS_PATH}"', shell=True)
with open("key.key", "rb") as f:
    key = f.read()
cipher = Fernet(key)
def encrypt(data):
    return cipher.encrypt(data)
def decrypt(data):
    return cipher.decrypt(data)
def split_data(data, size=1024*1024):
    return [data[i:i+size] for i in range(0, len(data), size)]
@app.route('/')
def home():
    return render_template('index.html')
@app.route('/upload', methods=['POST'])
def upload():
    file = request.files['file']
    filename = re.sub(r'\s+', '_', file.filename)
```

```
    data = file.read()
    encrypted = encrypt(data)
    blocks = split_data(encrypted)
    for i, block in enumerate(blocks):
        block_name = f"{filename}_part{i}"
        with open(block_name, 'wb') as f:
            f.write(block)
        os.system(f'hdfs dfs -put "{block_name}" "{HDFS_PATH}"')
        os.remove(block_name)
    return f"Uploaded: {filename}"
@app.route('/list')
def list_files():
    output = subprocess.getoutput(f'hdfs dfs -ls "{HDFS_PATH}"')
    lines = output.split("\n")
    files = set()
    for line in lines:
        if "_part" in line:
            name = line.split("/")[-1]
            original = name.split("_part")[0]
            files.add(original)
    return "\n".join(sorted(files))
@app.route('/download/<filename>')
def download(filename):
    output = subprocess.getoutput(f'hdfs dfs -ls "{HDFS_PATH}"')
    lines = output.split("\n")
    blocks = []
    for line in lines:
```

```
        if filename in line:
            parts = line.split()
            blocks.append(parts[-1])
    blocks.sort(key=lambda x: int(x.split("_part")[-1]))
    data = b''
    for block in blocks:
        os.system(f'hdfs dfs -get "{block}" .')
        local_file = block.split("/")[-1]
        with open(local_file, 'rb') as f:
            data += f.read()
        os.remove(local_file)
    try:
        decrypted = decrypt(data)
    except:
        return "Decryption failed (possible corruption or key mismatch)"
    output_file = "download_" + filename
    with open(output_file, 'wb') as f:
        f.write(decrypted)
    return send_file(output_file, as_attachment=True)
@app.route('/delete/<filename>')
def delete(filename):
    output = subprocess.getoutput(f'hdfs dfs -ls "{HDFS_PATH}"')
    lines = output.split("\n")
    for line in lines:
        if filename in line:
            filepath = line.split()[-1]
            os.system(f'hdfs dfs -rm "{filepath}"')
```

```
    return f"Deleted: {filename}"

if __name__ == '__main__':

    app.run(host='0.0.0.0', port=5000)
```

7.2 Style.css

```
body {
    font-family: Arial, sans-serif;
    background: #f4f6f8;
    text-align: center;
}

h1 {
    background: #2c3e50;
    color: white;
    padding: 15px;
}

.container {
    width: 60%;
    margin: auto;
    background: white;
    padding: 20px;
    margin-top: 20px;
    border-radius: 10px;
    box-shadow: 0 0 10px gray;
}

button {
    padding: 8px 15px;
    background: #3498db;
    color: white;
    border: none;
    cursor: pointer;
    border-radius: 5px;
}

button:hover {
    background: #2980b9;
}
```

```
input {
   padding: 5px;
   margin: 5px;
}
```

7.3 Index.html

```
<!DOCTYPE html>
<html>
<head>
<title>Cloud SaaS</title>
<link rel="stylesheet" href="/static/style.css">
</head>
<body>
<h1>Cloud Storage System (HDFS)</h1>
<div class="container">
<h3>Upload File</h3>
<form action="/upload" method="post" enctype="multipart/form-data">
<input type="file" name="file" required>
<button type="submit">Upload</button>
</form>
<hr>
<h3>List Files</h3>
<button onclick="loadFiles()">Show Files</button>
<ul id="fileList"></ul>
<hr>
<h3>Download File</h3>
<input type="text" id="downloadFile" placeholder="Enter filename">
<button onclick="downloadFile()">Download</button>
<hr>
<h3>Delete File</h3>
```

```
<input type="text" id="deleteFile" placeholder="Enter filename">
<button onclick="deleteFile()">Delete</button>
</div>
<script>
function loadFiles(){
fetch('/list')
.then(res=>res.text())
.then(data=>{
let list=document.getElementById('fileList');
list.innerHTML="";
data.split("\n").forEach(line=>{
if(line.trim()!==""){
let li=document.createElement("li");
li.innerText=line;
list.appendChild(li);
}
});
});
}
function downloadFile(){
let file=document.getElementById('downloadFile').value;
window.location.href="/download/"+file;
}
function deleteFile(){
let file=document.getElementById('deleteFile').value;
fetch("/delete/"+file)
.then(res=>res.text())
.then(data=>alert(data));
}
```

</script>

</body>

</html>

8.EXECUTION

7.1 Environment Setup

- Install a Linux-based operating system on all machines (physical/virtual)
- Ensure all machines are connected on the same network
- Install Java (JDK 8 or above)
- Verify Java installation using:
    - java -version
- Set JAVA_HOME environment variable on all machines

7.2 Hadoop Installation

- Download Apache Hadoop (version 3.x)
- Extract the Hadoop archive:
    - tar -xvzf hadoop-x.x.x.tar.gz
- Set Hadoop environment variables:
    - HADOOP_HOME
    - Update PATH with Hadoop binaries
- Verify installation using:
    - hadoop version

7.3 Network Configuration

- Assign unique IP addresses or hostnames to all nodes
- Ensure connectivity between nodes using:
    - ping <node>
- Update /etc/hosts file on all machines:
    - Map IP addresses to hostnames

7.4 Hadoop Configuration

- Configure core-site.xml:
  - Set fs.defaultFS to point to master node
- Configure hdfs-site.xml:
  - Define NameNode storage directory
  - Define DataNode storage directory
  - Set replication factor
- Configure workers file:
  - Add worker node hostnames/IPs

7.5 Starting HDFS Cluster

- Format the NameNode:
  - hdfs namenode -format
- Start HDFS services:
  - start-dfs.sh
- Start DataNode on worker machines:
  - hdfs --daemon start datanode
- Verify running processes:
  - jps
- Access HDFS web interface:
  - http://<master-node>:9870

7.6 HDFS Directory Setup

- Create user directory in HDFS:
  - hdfs dfs -mkdir -p /user/<username>
- Verify directory creation:
  - hdfs dfs -ls /user

7.7 Application Setup

- Install required Python libraries:
  - pip install flask cryptography
- Create project directory structure:
  - Controller file (controller.py)
  - Templates folder (index.html)
  - Static folder (style.css)
  - Encryption key file (key.key)
- Generate encryption key using Python

7.8 Running the Application

- Start the Flask server:
  - python controller.py
- Access the web interface using browser:
  - http://<master-node>:5000

9.OUTPUT

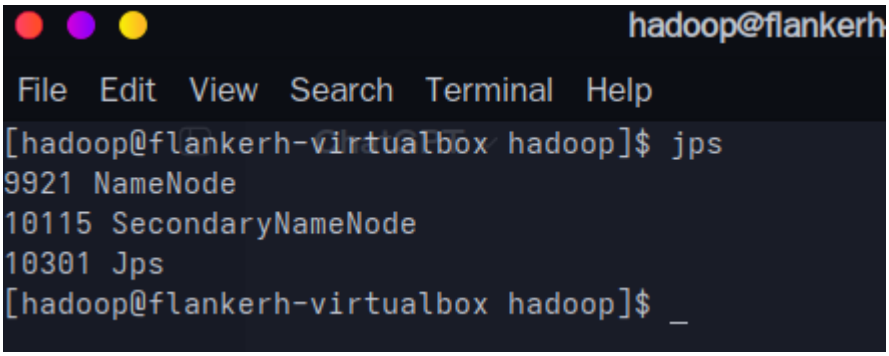

Fig 1 : Hadoop nodes running on NameNode

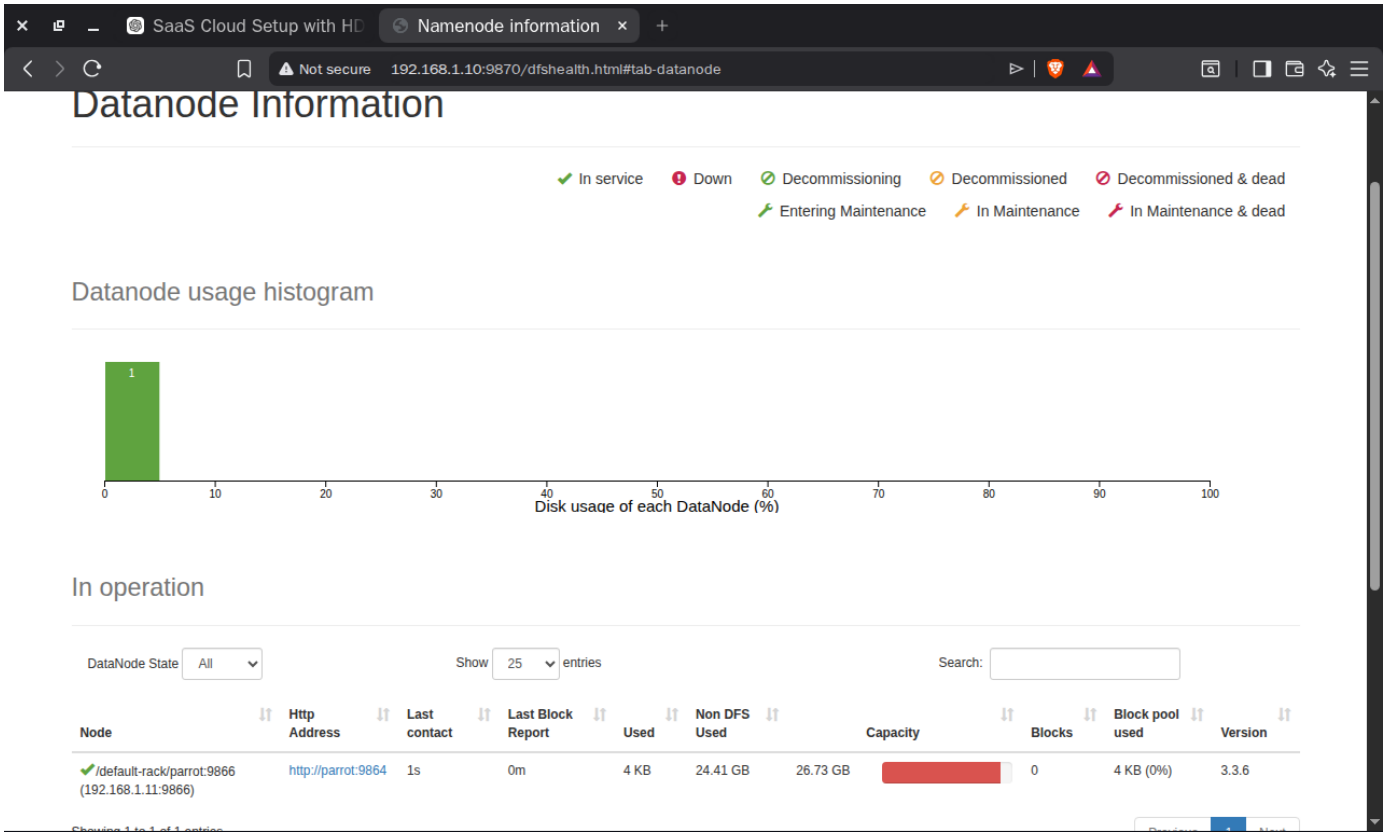

Fig 2 : Connected datanodes and their informations

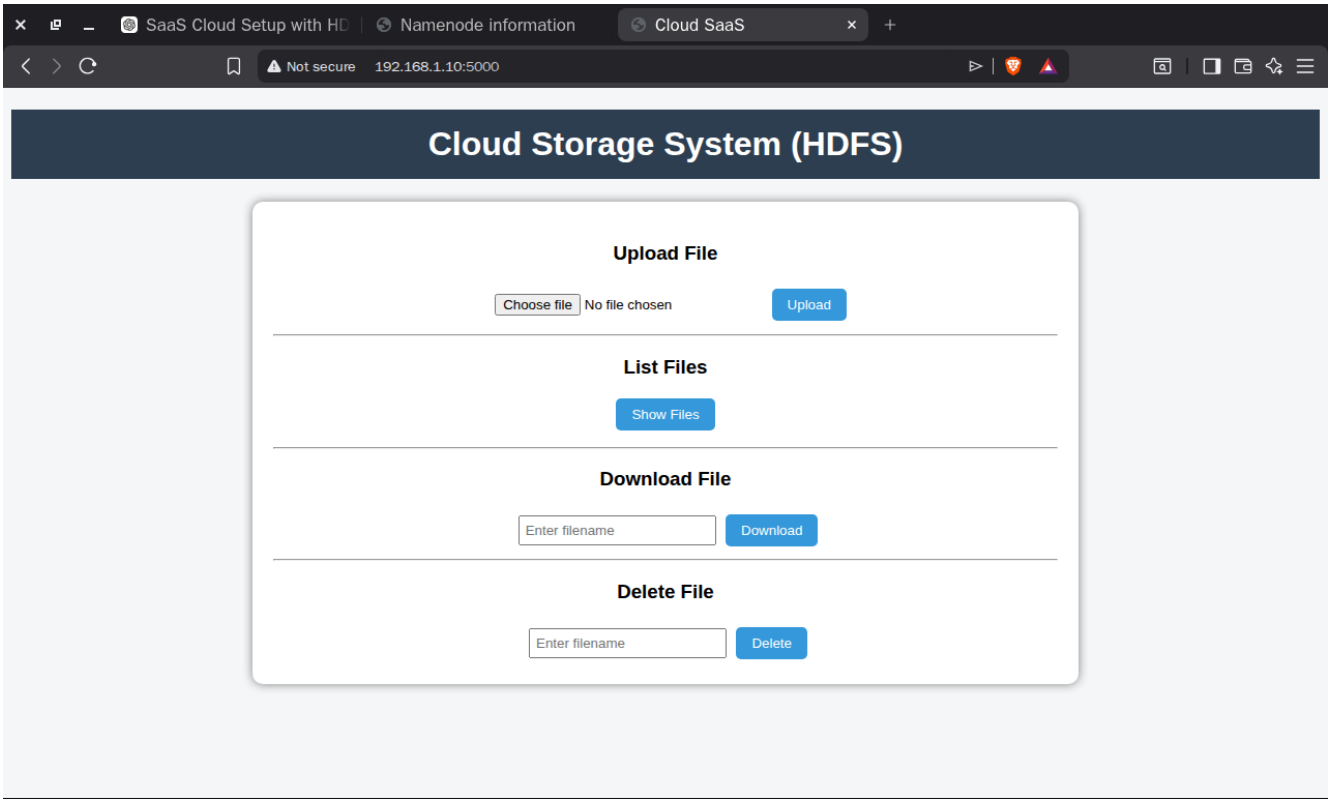

Fig 3 :User Interface

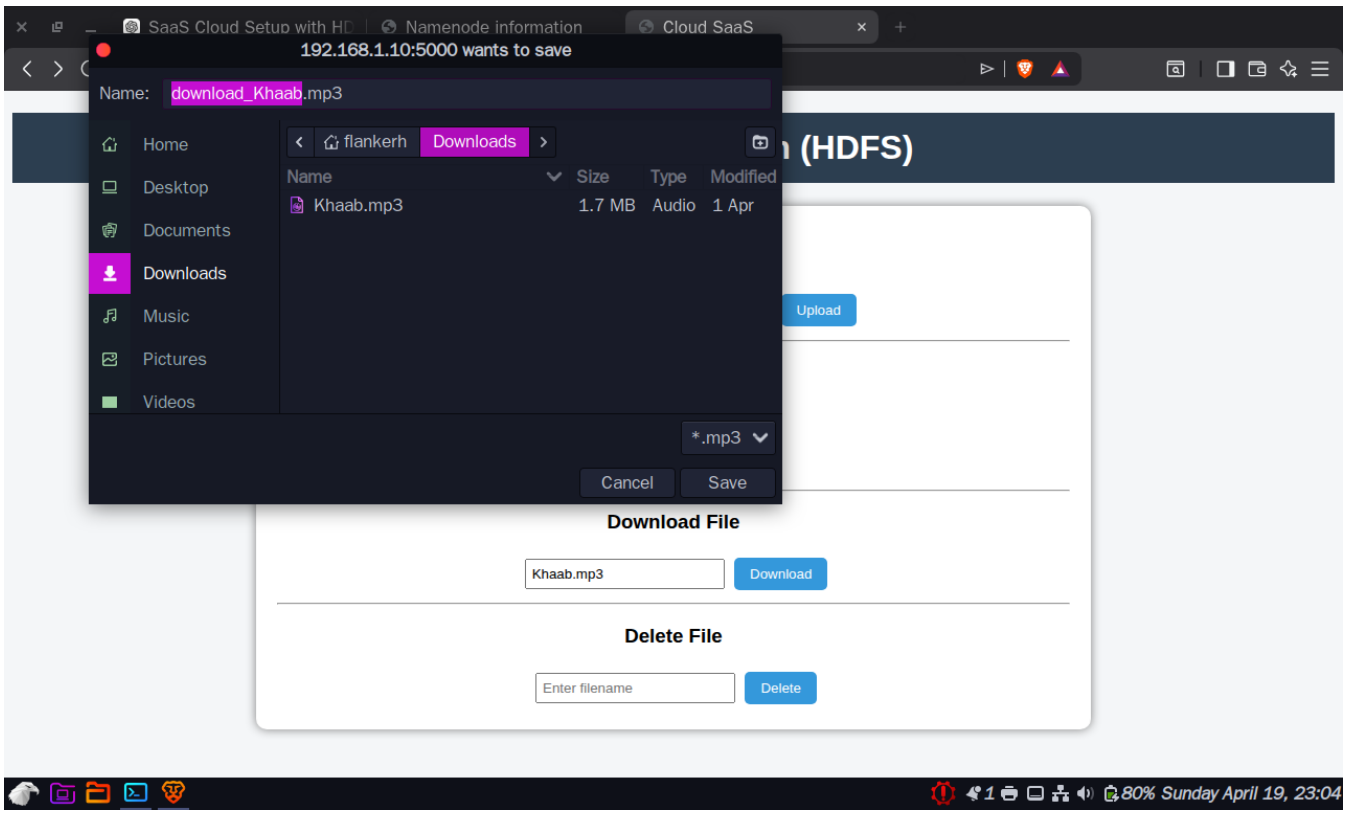

Fig 4: Downloading File

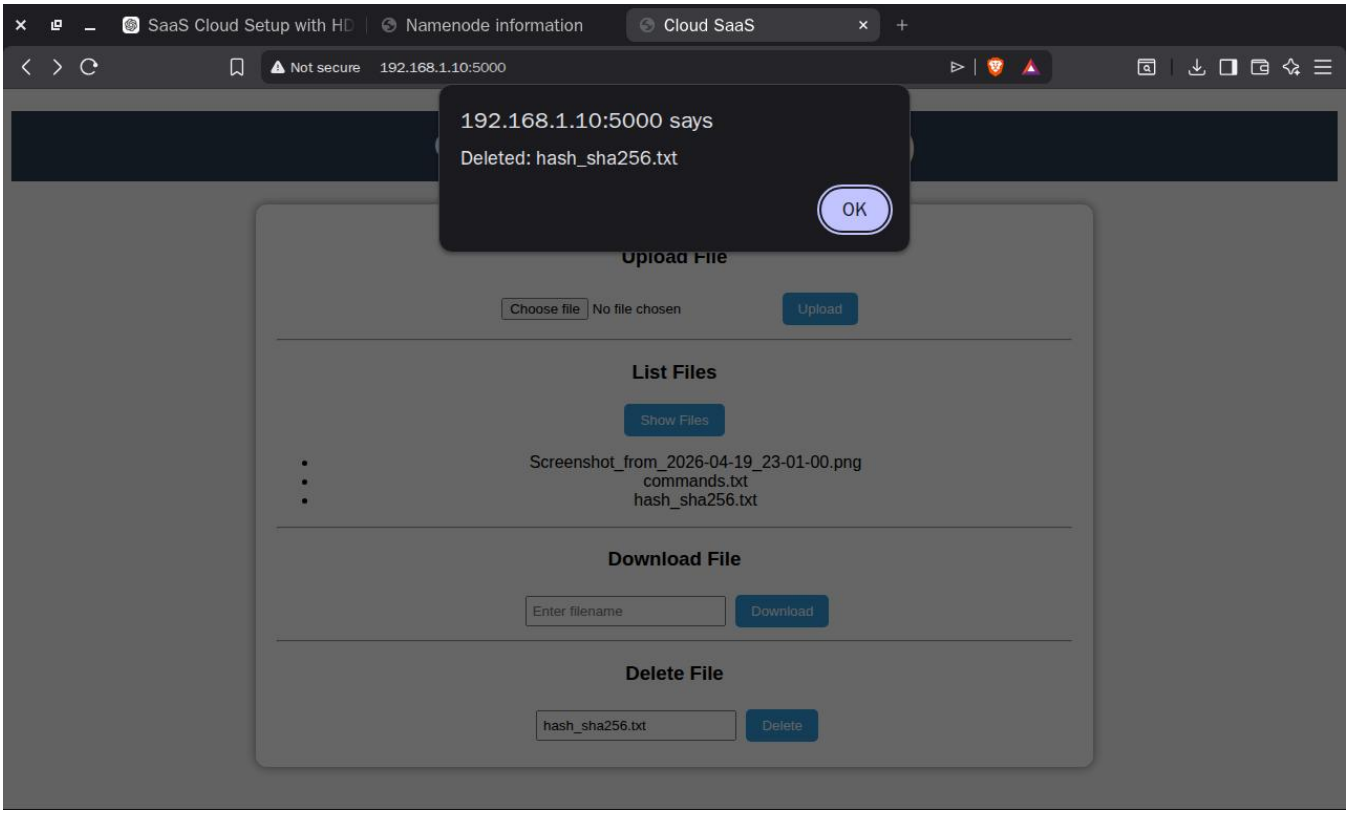

Fig 5: Deleting File

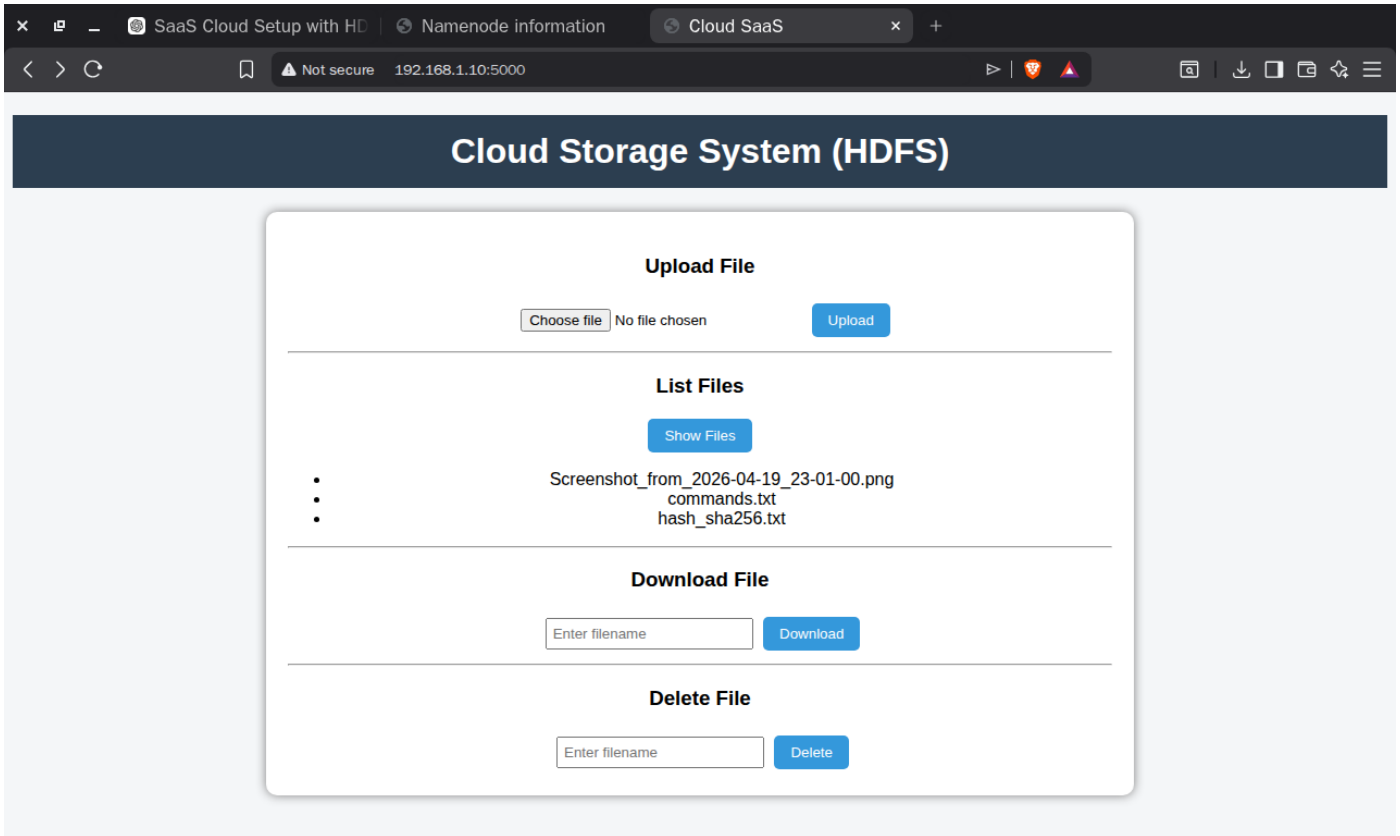

Fig 6: Listing all the files stored